THE KAFKA

ECOSYSTEM

Connect · Schema Registry
Kafka Streams · ksqlDB

Rajesh Pasham

Kafka Connect

Move data in & out of Kafka

Schema Registry

Govern message schemas & evolution

Kafka Streams

Java stream processing library

ksqlDB

SQL-powered event streaming database

Why the Kafka Ecosystem Exists

The patterns that emerge from production Kafka usage demand shared infrastructure solutions

The Danger of Rolling Your Own

As Kafka adoption grows, teams are tempted to build their own schema validation, data pipeline connectors, stream processing frameworks and state management.

This is "undifferentiated infrastructure" — critical code that every team must write, but delivers no unique business value.

The Kafka community recognised these patterns and built first-class components so you never have to:

- Kafka Connect — replaces custom ETL pipelines
- Schema Registry — replaces custom schema stores
- Kafka Streams — replaces custom processing frameworks
- ksqlDB — replaces custom SQL-over-Kafka layers

Kafka Connect

Pluggable connectors to 500+ external systems. Zero custom pipeline code.

Schema Registry

Centralised schema store with compatibility enforcement at runtime.

Kafka Streams

Java library for stateful stream processing — ships inside your app.

ksqlDB

SQL engine for continuous stream processing. Language-agnostic.

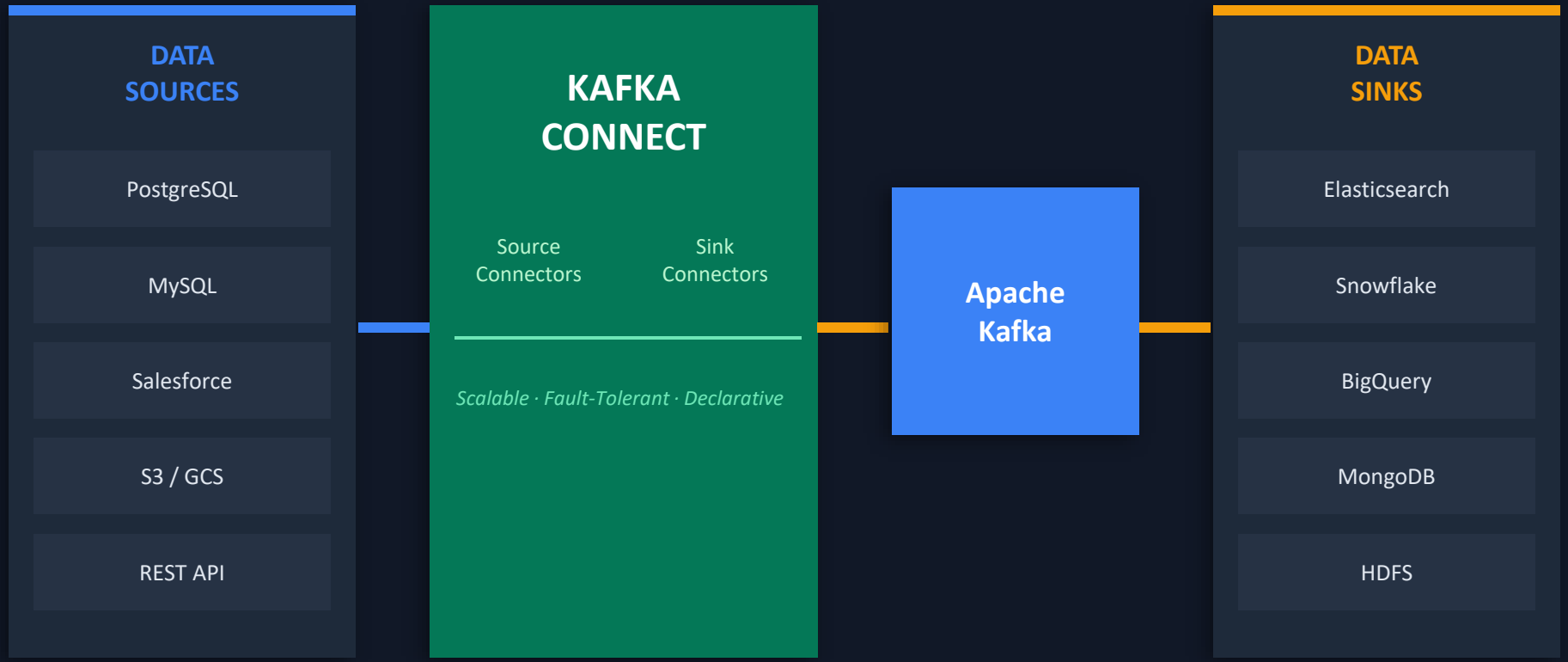
01

Kafka Connect

The data integration system that bridges Kafka with the rest of the world

Kafka Connect — The Integration Layer

Moving data between Kafka and every external system without custom code



Kafka Connect — Key Characteristics

Why Connect beats custom-built integration pipelines every time

External Process

Runs on dedicated worker nodes independent of Kafka brokers. Never competes with broker resources.

Horizontally Scalable

Add Connect workers to a cluster without downtime. Work is auto-redistributed across all workers.

Fault Tolerant

If a worker fails, tasks are reassigned to surviving workers. No manual intervention or data loss.

Declarative Config

Configure connectors via JSON posted to a REST endpoint. No code required — just describe what you want.

500+ Pre-Built Connectors

Confluent Hub hosts connectors for every major database, cloud service and SaaS platform — tested at scale.

Source & Sink Patterns

Source connectors act as producers. Sink connectors act as consumers. Kafka only ever sees standard clients.

Source vs Sink Connectors

Each connector pluggable as a JAR, activated as a runtime entity via REST

SOURCE CONNECTOR

Acts as a Kafka Producer

- Reads records from an external system (DB, API, filesystem)
- Converts records to Kafka messages and writes to a topic
- Handles offset tracking for exactly-once or at-least-once delivery
- Common examples: JDBC Source, Debezium CDC, HTTP Source
- The Kafka cluster only ever sees a regular producer

SINK CONNECTOR

Acts as a Kafka Consumer

- Subscribes to one or more Kafka topics
- Converts Kafka messages and writes to the external target
- Manages consumer offsets and delivery semantics
- Common examples: JDBC Sink, S3 Sink, Elasticsearch Sink
- The Kafka cluster only ever sees a regular consumer

02

Schema Registry

Governing the contracts between producers and consumers over time

Confluent Schema Registry

The centralised contract layer for all message formats in your Kafka cluster

Problem 1: New Consumers

New teams consuming existing topics need to understand the message format — but there's no built-in contract.

Problem 2: Schema Evolution

Business changes mean schemas change. Uncoordinated evolution silently breaks consumers downstream.

Standalone Server

Runs external to brokers. Stores schemas in an internal Kafka topic. Supports HA deployment for resilience.

Schema DB + Cache

Every schema gets an immutable integer ID. IDs cached in producers/consumers — no round trip overhead at steady state.

Compatibility Rules

Define BACKWARD, FORWARD or FULL compatibility per topic. Producer and consumer APIs enforce rules before any produce/consume.

Three Formats

JSON Schema, Apache Avro and Protocol Buffers. Each supports IDL source files enabling schema collaboration via pull requests.

How Schema Registry Works at Runtime

The API calls that prevent schema drift between producers and consumers

PRODUCER FLOW

1. Producer calls Schema Registry REST endpoint with message schema
2. Registry checks schema against compatibility rules for this topic
3. If compatible → returns schema ID, produce succeeds
4. If incompatible → returns error, application handles gracefully
5. Schema IDs cached locally — subsequent produces skip the round trip

CONSUMER FLOW

1. Consumer reads message (contains embedded schema ID)
2. Consumer checks schema ID against the version it expects
3. Registry confirms schema compatibility for this consumer
4. If incompatible → consumer API raises an exception, message skipped
5. Full schema evolution audit trail kept for debugging

Supported Schema Formats

JSON Schema · Apache Avro · Protocol Buffers — each with production-grade tooling

JSON Schema

Advantages

- Human-readable & widely understood
- Flexible — any JSON producer/consumer
- Great for REST-based integrations

Trade-offs

- Verbose for large payloads
- No built-in code generation

Best for: REST APIs, web service integrations, heterogeneous consumer environments

Apache Avro

Advantages

- Compact binary — up to 10x smaller than JSON
- Rich IDL (.avsc) with Maven/Gradle plugins
- Most popular format in the Confluent ecosystem

Trade-offs

- Requires schema to decode
- Primarily Java-centric tooling

Best for: High-throughput internal pipelines, Java microservices, analytics workloads

Protocol Buffers

Advantages

- Fastest serialization performance
- Excellent polyglot support (Go, Python, C++)
- Strong backward/forward compatibility

Trade-offs

- More complex toolchain setup
- Less Kafka-specific tooling than Avro

Best for: Multi-language environments, gRPC integrations, low-latency requirements

03

Kafka Streams

A Java stream processing library that lives inside your applications

Kafka Streams — Stream Processing as a Library

All the power of distributed stream processing, zero extra infrastructure

The Library Philosophy

Kafka Streams is a Java API — not a dedicated cluster you must deploy and manage.

Add it as a dependency and it runs inside your existing application process — Spring Boot, Micronaut or standalone JAR.

This means:
- Deploy with your app — no separate cluster to provision
- Scale with your consumer group — add JVM instances, partitions auto-redistribute
- No new infrastructure to secure or monitor

It's the cleanest architectural integration point for stream processing.

Functional Java API

Filter, map, flatMap, branch — clean functional primitives for transforming event streams.

Filter · Group · Aggregate · Join

All standard stream processing operations built-in. No framework code needed.

Fault-Tolerant State

State on RocksDB (local disk) AND replicated to internal Kafka topics. Survives node failures.

Consumer Group Scaling

A Streams app IS a consumer group. Add instances → Kafka redistributes partitions automatically.

State Management in Kafka Streams

How Streams solves the hardest problem in distributed stream processing

The State Problem

Any interesting stream processing — aggregation, windowing, joining — requires stateful memory. If your process restarts, that state is gone. Managing this reliably at scale is fiendishly complex.

1. Off-Heap Local Storage (RocksDB)

State persisted to a local RocksDB instance on disk. Not in JVM heap — no GC pressure. On restart, state restored instantly from local disk (same machine, high-bandwidth access). Far faster than any remote database lookup.

2. Replicated to Internal Kafka Topics

Every local state store is continuously backed up to a dedicated changelog topic in Kafka. If the physical node is lost entirely (disk failure), state rebuilds from the Kafka changelog. Also used during consumer group scale-out rebalancing.

3. Interactive Queries — Built-In Materialised View

The RocksDB state store is queryable from outside the Streams application. Your service can expose a REST API reading directly from its own stream-processed state — no separate database required. Turns a Streams app into a live materialised view.

Real-World Streams Example

A shipment notification microservice built with Kafka Streams

Product Catalog

Shipment Events

Customer Records

Shipment Notification Service (Kafka Streams)

```
JOIN: shipments + products
JOIN: + customer records
FILTER: in-transit only
WINDOW: 15-min aggregations
EXPOSE: REST /status API
```

Email & SMS Notifications
→ notification-events topic

REST API /shipment/{id}
Interactive Queries (no DB!)

04

ksqlDB

SQL-powered continuous stream processing for any programming language

ksqlDB — The Event Streaming Database

When you need continuous processing without committing to a Java microservice

What is ksqlDB?

ksqlDB is a purpose-built stream processing database for Apache Kafka. It exposes a SQL dialect for defining continuous queries over Kafka topics.

Unlike Kafka Streams (a Java library), ksqlDB is a standalone server cluster. You send SQL statements to it via REST or CLI — making it fully language-agnostic.

When your job has no functions beyond 'read topic → compute → write topic', ksqlDB is the cleanest choice: no application code, just SQL.

Runs on its own fault-tolerant cluster adjacent to — but independent of — the Kafka cluster.

Stream Processing in SQL

Continuous queries that transform, filter, join and aggregate Kafka topics using familiar SQL syntax.

Scalable Fault-Tolerant Cluster

Runs on its own cluster outside Kafka, scaling horizontally like any consumer group.

REST API + CLI

Language-agnostic. Any stack can submit queries. Shell available for development and debugging.

Java Client Library

An idiomatic Java wrapper provides programmatic access for Java-based integrations.

Kafka Connect Integration

CREATE CONNECTOR statements manage Connect connectors directly inside the SQL environment.

ksqlDB in Practice — Key SQL Constructs

Streams, tables, aggregations and joins — all in SQL

Create a Stream

```
CREATE STREAM orders (
  order_id  VARCHAR,
  customer_id VARCHAR,
  amount    DOUBLE
) WITH (
  KAFKA_TOPIC='orders-topic',
  VALUE_FORMAT='AVRO'
);
```

Create a Table (Materialised View)

```
CREATE TABLE order_totals AS
  SELECT customer_id,
    SUM(amount) AS total
  FROM orders
  GROUP BY customer_id
  EMIT CHANGES;
```

Pull Query (Point-in-Time)

```
SELECT * FROM order_totals
  WHERE customer_id = 'C-1042';
```

Push Query (Continuous)

```
SELECT order_id, amount
  FROM orders
  WHERE amount > 1000
  EMIT CHANGES;
```

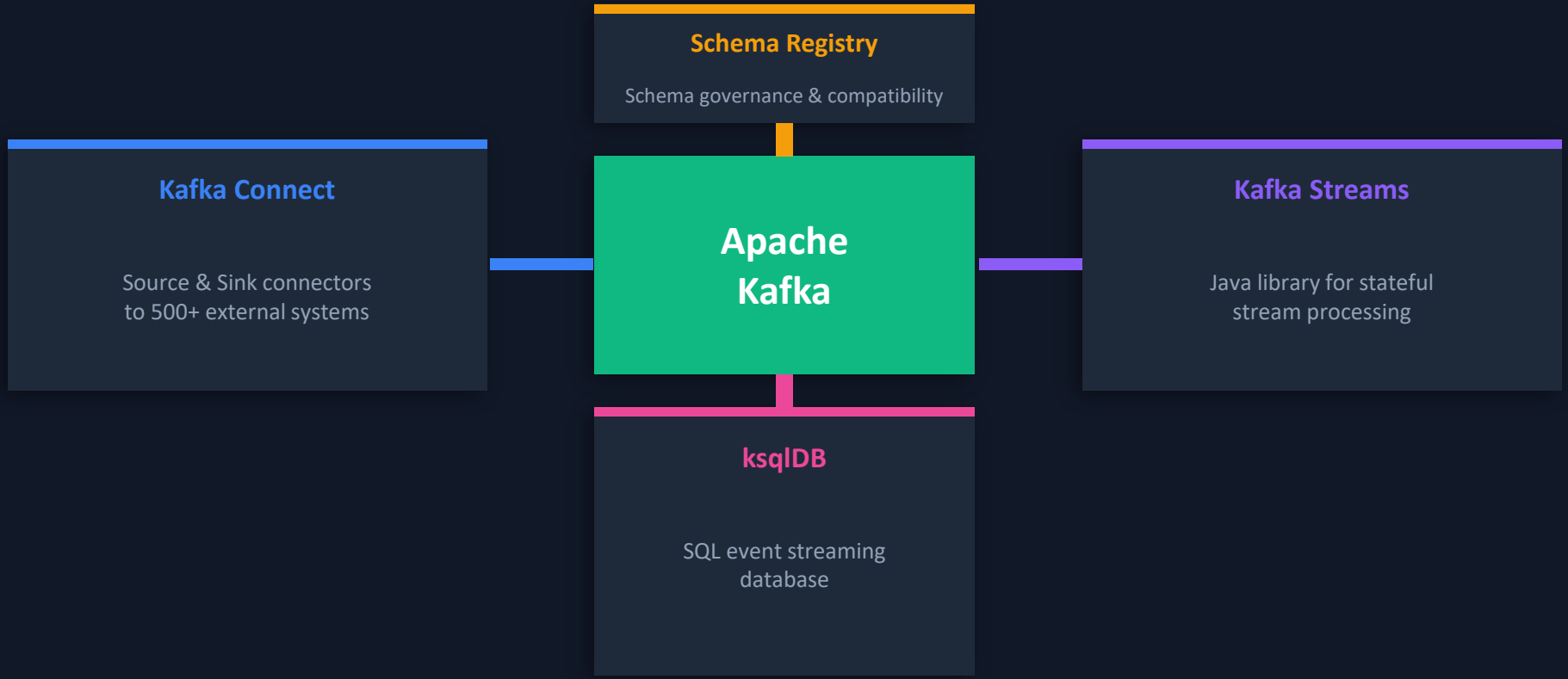
Kafka Streams vs ksqlDB — Choosing the Right Tool

Both process Kafka events — the difference is deployment model and interface

Dimension	Kafka Streams	ksqlDB
Deployment	Library inside your app (JAR dependency)	Standalone server cluster
Language	Java only	Language-agnostic (REST / CLI)
Interface	Functional Java API (DSL + Processor API)	SQL dialect
State Store	RocksDB, local disk + Kafka changelog	RocksDB, local disk + Kafka changelog
Scaling	Add JVM instances → consumer group rebalance	Add ksqlDB nodes → rebalance
Co-location	Shares process with REST APIs, Spring Boot…	Separate cluster, no co-location
Connect integration	Via separate Connect cluster	CREATE CONNECTOR built-in
Best for	Complex microservices with side effects	Pure ETL / analytics jobs in SQL

The Confluent Ecosystem Map

How all four components interconnect around Apache Kafka



Key Takeaways

01	**Kafka Connect removes integration boilerplate**	500+ connectors mean zero custom pipeline code. Source and sink connectors scale and fail independently of your applications.
02	**Schema Registry is non-negotiable at scale**	Any non-trivial Kafka system needs governed schema evolution. Avro for high-throughput; JSON Schema for REST integrations.
03	**Kafka Streams is a library, not infrastructure**	Deploy inside your microservice. State is fault-tolerant via RocksDB + changelog topics. Scale with your existing consumer group.
04	**ksqlDB is SQL for continuous processing**	Pure ETL or analytics jobs without a Java app. ksqlDB's REST/CLI makes stream processing accessible to any language.
05	**The ecosystem exists to eliminate undifferentiated code**	Every component exists because teams kept solving the same problem. Leverage community work — focus on what makes your business unique.